

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Mock Interview

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA0510

Slot A

COURSE OUTCOME COVERED

CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)

Assessment Tool Weightage: 10%

QUESTIONS:4

Total Marks 20

Duration :60 Minutes Annexure A

Mock Interview

Code of Conduct:

I _____ M.THANUJA(192425356)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M.THANUJA

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge	
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problemsolving ability; requires guidance	Unable to solve or apply concepts	
Analytical Thinking	15	Analyzes questions critically and provides wellstructured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated	
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively	
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism	

Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately	
-----------------------	----	--	---	------------------------------------	---------------------------------	--

Image & Video Processing

1. A video stream drops frames during processing. Analyze the problem and redesign your approach to ensure smooth playback, explaining trade-offs and optimization techniques.

1. VIDEO STREAM DROPS FRAMES DURING PROCESSING

Problem Understanding

The video stream is dropping frames because the processing time is high. The system must be redesigned to provide smooth playback while maintaining acceptable accuracy.

Causes

1. High-resolution video.
2. Processing every frame.
3. Complex image-processing operations.
4. Slow detection algorithms.
5. Limited CPU or memory.
6. Inefficient processing pipeline.

Proposed Pipeline

Video Input → Frame Capture → Resize → Frame Selection → Processing → Display

Solution

1. Reduce Resolution

Resize frames to a smaller resolution to reduce computation.

2. Frame Skipping

Process only necessary frames instead of every frame.

3. Region of Interest

Process only the required area of the frame.

4. Efficient Algorithms

Use lightweight OpenCV operations and efficient detection methods.

5. Buffering

Separate frame capture and processing so that processing delays do not stop video acquisition.

6. Remove Unnecessary Operations

Avoid repeated or unnecessary image-processing operations.

Trade-Off

High resolution and processing every frame provide better accuracy but require more processing time.

Lower resolution and frame skipping improve speed but may reduce accuracy and miss short events.

Justification

Reducing the image size and processing workload decreases CPU usage and processing delay. ROI and efficient algorithms further improve real-time performance.

Conclusion

The redesigned system provides smoother playback by using **frame resizing, frame skipping, ROI processing, buffering, and efficient algorithms.**

2. You need to process only every 5th frame of a video. Design an efficient solution and justify your approach, considering performance and accuracy.

2. PROCESS ONLY EVERY 5TH FRAME

Problem Understanding

The requirement is to process only every fifth frame to reduce computational workload while maintaining acceptable video analysis performance.

Proposed Approach

Use a frame counter and process a frame whenever the frame number is divisible by 5.

Pipeline

Video Input → Read Frame → Increment Counter → Check MOD 5 → Process/Skip → Display

Pseudocode

BEGIN

Open video

frame_count = 0

WHILE video is available DO

 Read frame

 IF frame is unavailable THEN

 BREAK

 END IF

 frame_count = frame_count + 1

 IF frame_count MOD 5 = 0 THEN

 Process frame

 Detect required objects/features

 Display processed result

 ELSE

 Skip frame

 END IF

END WHILE

Release video

Close windows

END

Example

Frame 1 → Skip
Frame 2 → Skip
Frame 3 → Skip
Frame 4 → Skip
Frame 5 → Process

Frame 6 → Skip
Frame 7 → Skip
Frame 8 → Skip
Frame 9 → Skip
Frame 10 → Process

Performance

If a video contains 100 frames:

Normal processing: 100 frames

Every 5th frame: Approximately 20 frames

Therefore, the processing workload is greatly reduced.

Trade-Off

Advantage:

- Lower CPU usage
- Faster processing
- Lower computational cost

Disadvantage:

- Some fast movements may be missed.
- Temporal accuracy is reduced.

Justification

Processing every fifth frame is suitable when the scene does not change rapidly and computational efficiency is more important than analyzing every frame.

Conclusion

Using $\text{frame_count} \bmod 5 = 0$ provides a simple and efficient solution for processing only every fifth frame.

3. A system must process both images and videos using the same pipeline. Propose a generalized design and explain how it handles both cases effectively.

3. PROCESS IMAGES AND VIDEOS USING THE SAME PIPELINE

Problem Understanding

The system must process both images and videos using a common computer vision pipeline. An image is a single frame, whereas a video consists of multiple frames.

Proposed Pipeline

Input → Identify Input Type → Frame Acquisition → Preprocessing → Detection → Analysis → Output

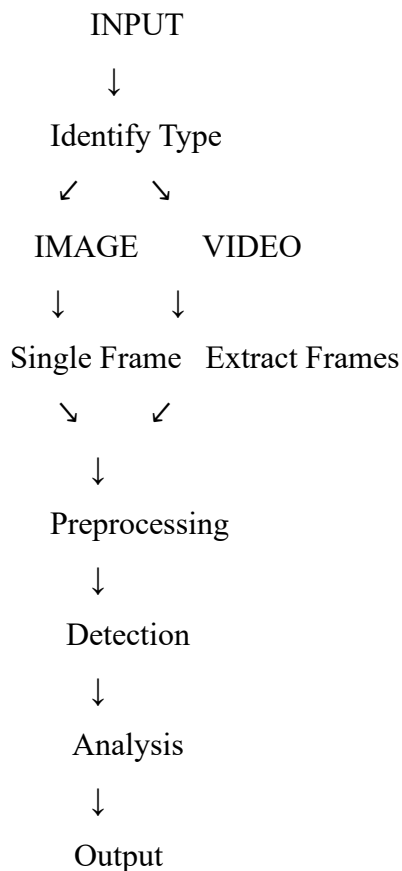


Image Processing

Read Image



Preprocess

↓
Detect
↓
Analyze
↓
Display Result

Video Processing

Open Video

↓
Read Frame

↓
Preprocess

↓
Detect

↓
Analyze

↓
Display

↓
Read Next Frame

Generalized Pseudocode

BEGIN

Get input

IF input is IMAGE THEN

Read image

Preprocess image

Detect objects/features

Analyze image

Display result

ELSE IF input is VIDEO THEN

Open video

WHILE video has frames DO

Read frame

Preprocess frame

Detect objects/features

Analyze frame

Display result

END WHILE

Release video

END IF

END

Advantages

1. Code reusability.

2. Easy maintenance.
3. Same processing logic.
4. Consistent results.
5. Reduced code duplication.

Justification

The common modules can be reused for both image and video processing. The only major difference is that an image is processed once, while video processing repeatedly applies the same pipeline to each frame.

Conclusion

A modular generalized pipeline efficiently supports both **images and videos** by treating each video frame as an individual image.

4. You need to extract frames from a video and save only those with significant changes.

Analyze the problem and design a method, explaining your decision criteria and approach.

4. EXTRACT AND SAVE FRAMES WITH SIGNIFICANT CHANGES

Problem Understanding

The objective is to extract frames from a video and save only frames that contain significant visual changes.

Proposed Pipeline

Video → Frame Acquisition → Grayscale → Noise Removal → Frame Difference → Thresholding → Change Measurement → Decision → Save Frame

Solution

First, read the first frame and use it as the reference frame.

For every subsequent frame:

1. Convert it to grayscale.
2. Apply Gaussian blur.
3. Compare it with the previous frame.
4. Calculate the difference.
5. Apply thresholding.

6. Calculate the percentage of changed pixels.
7. Compare the change with a predefined threshold.
8. Save the frame if the change is significant.

Pseudocode

BEGIN

Open video

Read first frame

Convert first frame to grayscale

Apply Gaussian blur

previous_frame = first frame

WHILE video has frames DO

Read current frame

Convert current frame to grayscale

Apply Gaussian blur

Calculate absolute difference between
current_frame and previous_frame

Apply thresholding

Calculate changed pixel percentage

IF changed pixel percentage > threshold THEN

Save current frame

END IF

previous_frame = current frame

END WHILE

Release video

END

Decision Criteria

Change $\leq$ Threshold

↓

Ignore Frame

Change > Threshold

↓

Save Frame

Change Percentage

Change Percentage = (Changed Pixels / Total Pixels) $\times$ 100

Noise Handling

Gaussian blur helps remove small noise and insignificant variations before calculating frame differences.

Trade-Off

Low threshold:

- More frames are saved.
- Higher sensitivity.
- More unnecessary frames may be stored.

High threshold:

- Fewer frames are saved.

- Less storage is required.
- Small meaningful changes may be missed.

Justification

Frame differencing identifies changes between consecutive frames, while thresholding determines whether the change is significant. This reduces unnecessary frame storage while preserving important changes.

Conclusion

The proposed method efficiently extracts and saves only **significant-change frames**, balancing storage efficiency and detection accuracy.